

Full Stack Developer Case Study

Project: Mini ERP + CRM Operations Portal

Deadline

48 hours from the time this assignment is shared.

Business Context

You are building a small ERP/CRM system for a wholesale/distribution company.

The company deals with customers, products, stock, purchase orders, sales challans, invoices, and basic CRM follow-ups. The system will be used by internal employees such as sales, warehouse, and accounts teams.

The goal is not to build a huge system, but to show that you understand full-stack development, backend APIs, database design, frontend UI, deployment, and real-world business flow.

Required Tech Stack

Backend

- Node.js
- TypeScript
- Express.js or NestJS
- PostgreSQL or MySQL
- REST APIs
- Proper validation and error handling

Frontend

- React
- HTML
- CSS
- JavaScript/TypeScript
- Responsive UI

Deployment / DevOps

- AWS deployment preferred
- Server setup should be documented
- Environment variables should be used

- GitHub repository with proper commits
 - README with setup instructions
-

Core Modules Required

1. Authentication and Roles

Create login functionality with role-based access.

Required roles:

- Admin
- Sales
- Warehouse
- Accounts

Simple JWT-based authentication is acceptable.

2. Customer CRM Module

Create a customer management section.

Each customer should have:

- Customer name
- Mobile number
- Email
- Business name
- GST number, optional
- Customer type: Retail, Wholesale, Distributor
- Address
- Status: Lead, Active, Inactive
- Follow-up date
- Notes

Required features:

- Add customer
 - Edit customer
 - Search customer
 - View customer detail page
 - Add follow-up notes
-

3. Product and Inventory Module

Create a product and stock management section.

Each product should have:

- Product name
- SKU/code
- Category
- Unit price
- Current stock
- Minimum stock alert quantity
- Location/warehouse

Required features:

- Add product
- Edit product

Stock movement log should track:

- Product
 - Quantity changed
 - Movement type: IN or OUT
 - Reason
 - Created by
 - Timestamp
-

4. Sales Challan Module

Create a sales challan flow.

A sales user should be able to:

- Select customer
- Add multiple products
- Add quantity for each product
- Generate challan number automatically
- Save challan as Draft or Confirmed

Important business logic:

- If challan is confirmed, stock should be reduced.
- Stock should not go negative.
- If stock is insufficient, API should return a proper error.
- Challan should store product snapshot data, not only product ID.

Challan fields:

- Challan number
 - Customer
 - Products
 - Total quantity
 - Status: Draft, Confirmed, Cancelled
 - Created by
 - Created date
-

API Expectations

Backend should include clean REST APIs.

Examples:

- POST /auth/login
- GET /customers

APIs should include:

- Input validation
 - Proper HTTP status codes
 - Error messages
 - Pagination where needed
 - Search/filter where needed
-

Frontend Expectations

Create a clean admin-style UI.

AWS / Deployment Expectations

Deployment should be done using any free hosting platform.

Acceptable options:

- Frontend: Vercel, Netlify, Render Static Site, or similar
- Backend: Render, Railway, Fly.io, or similar
- Database: Supabase, Neon, Render Postgres, or similar

AWS deployment is optional and will be treated as a bonus.
The candidate is not expected to spend money for this assignment.

If the candidate chooses not to deploy, they must provide:

- A working local setup
- A screen recording of the full flow
- Postman collection
- Clear README instructions

You must document:

- How the server was set up
- How environment variables are managed
- How to run the project locally
- How to deploy the project
- Any assumptions made

Bonus Points

Bonus features are not mandatory but will be appreciated:

- Docker setup
- GitHub Actions deployment
- Export invoice as PDF
- Upload product image to AWS S3

Submission Requirements

Submit the following:

1. GitHub repository link
 2. Live frontend URL
 3. Live backend API URL
 4. Test login credentials for all roles
 5. Postman collection or API documentation
 6. README with setup and deployment instructions
 7. Short explanation of architecture
 8. Known limitations or incomplete parts
-